
Machine Learning for Healthcare Report

Project 1: Time-Series

Afonso Ferreira da Silva Domingues
24-936-114
aferreira@student.ethz.ch

Thösam Norlha-Tsang
20-828-075
tnorlha@student.ethz.ch

Rahul Kaundal
19-931-468
rkaundal@student.ethz.ch

1 Data Processing and Exploration

Exploratory Data Analysis By plotting the feature distributions of our training set, we observe a range of distribution types across the data. Some features, such as *Na*, follow a normal, bell-shaped distribution, while others, like *Bilirubin*, exhibit skewness. One feature, *MechVent*, consistently takes the value 1. Additionally, several features are categorical, including *Gender*, *ICUType*, *GCS*, and *MechVent*. The plots also reveal the presence of unrealistic values in some variables, leading to distributions with long tails, such as for *Temp*, and plots that are challenging to interpret, like for *pH*. To allow us to fully interpret these distorted plots, we temporarily crop these unrealistic values out of the visualization (without removing them). However, the outliers in the distribution will be removed in the preprocessing section.

Additionally, differences in value distributions can be observed across various groups. For instance, in terms of gender, the height and weight distributions differ, with the male population exhibiting a slight rightward shift compared to the female population. Regarding age, individuals in the (20, 30] group tend to have lower average cholesterol with less variance. For *ICUType*, patients in the Coronary Care and Cardiac Surgery Recovery Units have a narrower age spread compared to other units, despite similar mean and median ages. Other features show comparable distributions across groups.

Preprocess Data for Machine Learning For scaling the data we chose the `RobustScaler` rather than the `StandardScaler`. By inspecting the plots, we see that most of the features are not normally distributed. Also, it is highly likely that the data contains outliers, because occasionally there is faulty data as we can see in the temperature measurements. Hence, choosing `RobustScaler` is a valid option here.

2 Supervised Learning

Feature Set	Random Forest		Logistic Regression	
	AuROC	AuPRC	AuROC	AuPRC
Mean	0.811	0.473	0.823	0.476
Max	0.796	0.397	0.809	0.412
Last Measured	0.840	0.507	0.845	0.507

Table 1: Performance comparison of feature aggregation methods.

Classic Machine Learning Methods First, we compared different feature aggregation methods for each patient, namely taking the mean, the maximum, and the last measured value. From these results in Table 1, we observe that using the **last measured** value for each record yields the best performance in both models. Additionally, Logistic Regression and Random Forest achieve similar performances, with slight variations depending on the feature aggregation strategy.

Additional Features To better capture the temporal aspect of the data, we categorized the features into two distinct groups [1]:

- **Dynamic features:** These include blood pressure, heart rate, and glucose levels, which fluctuate significantly over time. To capture their variations, we compute the minimum, maximum, median, first, and last recorded values.
- **Laboratory features:** This group consists of biochemical lab values such as Albumin, ALT, AST, and Bilirubin, which tend to change less frequently. Since lab tests are typically performed at specific time points, we extract only the first and last recorded values.

We observe from Table 2 that adding these additional features improves the performance of the Random Forest model. However, Logistic Regression’s performance does not change significantly with the addition of the new features.

Pros and Cons of used classifiers From the results, we observe that Random Forest is better suited for leveraging the newly extracted features. This is likely because it can model complex, non-linear relationships - being more robust to useless features, whereas Logistic Regression, being a linear model, is limited in its ability to utilize richer feature sets effectively.

Recurrent Neural Networks Bidirectional RNNs process data in both forward and backward directions, enabling them to incorporate past and future information. This is valuable in ICU data analysis, providing a fuller view of a patient’s health trajectory. Unlike traditional RNNs, which rely only on past data for mortality prediction, BRNNs can consider future interventions, improving predictions.

The comparison between BiLSTM and LSTM (see Table 2) with identical hyperparameters shows similar performance, with BiLSTM slightly outperforming LSTM. Despite high AUROC scores, the AUPRC scores are moderate, suggesting the BiLSTM’s bidirectional structure captures more temporal dependencies, but does not significantly improve performance.

Transformers When comparing Transformers to RNNs, Transformers offer several key advantages. First, the self-attention mechanism of Transformers enables them to capture long-range dependencies more effectively than RNNs, which often struggle with this due to their sequential nature. While RNNs process sequences step-by-step, Transformers process them in parallel, significantly accelerating training times. Lastly, Transformers avoid the vanishing gradient problem, a common issue in RNNs that hampers learning over long sequences.

	RF	LR	LSTM	BiLSTM	Transf. w/ imp.	Transf. w/o imp.
AuROC	0.843	0.847	0.8329	0.8394	0.7882	0.7343
AuPRC	0.522	0.505	0.4869	0.4952	0.3662	0.2772

Table 2: Performance comparison of different machine learning models: Random Forest (RF) with additional features, Logistic Regression (LR) with additional features, LSTM, Bidirectional LSTM (BiLSTM) and Transformer with and without imputation.

Tokenizing Time-Series Data and Transformers We have trained two Transformers. The first Transformer was trained on imputed data. For imputing the data, we forward filled the missing value and filled up the remaining values with the median. The second Transformer was trained on encoded measurements, where each measurement was encoded as a triplet: time, variable, value.

In Table 2 we can see that the Transformer trained on the imputed data performed significantly better than the Transformer trained on the encoded data. The reason why both Transformer models performed poorly might be due to the usage of the multiheaded self-attention mechanism, which is not very good at capturing temporal dependencies [3].

3 Representation Learning

Pretraining and Linear Probes We used a Transformer to generate patient embeddings through self-supervised learning, using the data as a time grid representation. A logistic regression model was then trained on these embeddings. The results on test set C were worse than in part 2, indicating that with more labeled patients, the supervised Transformer performs better (see Table 3, row with 4000 labeled patients, which corresponds to the full training set A).

Simulating Label Scarcity The results show that logistic regression on pretrained embeddings outperforms the Transformer model when labeled data is very scarce. However, as the number of labeled patients increases, the Transformer model quickly improves, and eventually outperforms logistic regression on the pretrained embeddings. This suggests that while self-supervised pretraining is valuable in low-data settings, its advantage diminishes as more labeled data becomes available. The choice between pretraining and direct supervised learning therefore depends on data availability, with pretrained embeddings offering a strong baseline in label-scarce settings.

Visualising Learned Representations The data points with different labels are not well-separated in the reduced 2D space. Both t-SNE and UMAP visualizations indicate overlap between the classes, suggesting that the learned embeddings do not adequately separate the classes.

The low silhouette scores of **0.0284** for t-SNE and **0.0337** for UMAP further confirm that the quality of clustering in the reduced space is poor, with the data points from different labels not being distributed identically.

Labeled Patients	Transformer		Logistic Regression on Embeddings	
	AuROC	AUPRC	AuROC	AUPRC
100	0.5848	0.1885	0.6488	0.2263
500	0.7027	0.2765	0.7002	0.2889
1000	0.7306	0.2940	0.7307	0.3113
4000 (full training set A)	0.7975	0.3750	0.7475	0.3294

Table 3: Performance comparison on test set C with label scarcity and the full training set.

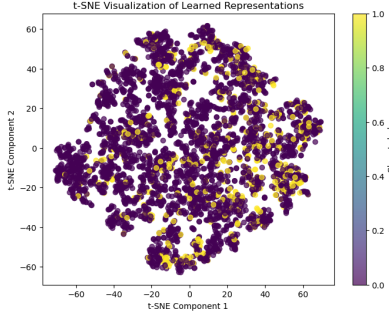


Figure 1: t-SNE visualization of representation learning embeddings

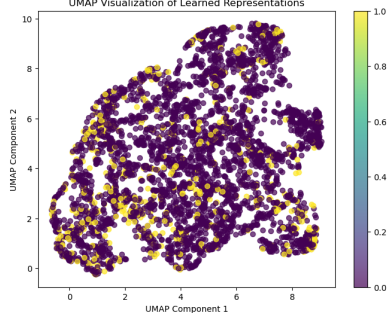


Figure 2: UMAP visualization of representation learning embeddings

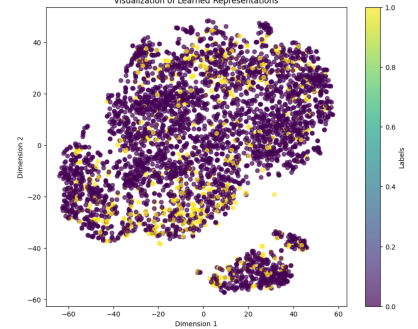


Figure 3: t-SNE visualization of LLM computed embeddings

4 Foundation Models

Prompting an LLM to solve a time-series problem We apply the feature engineering methods from Q2.1.2 in two different modalities: with and without scaling and imputation. After transforming the data, we identify the 20 features most correlated with the target variable in the training set. To prevent data leakage, we apply the same feature selection to the test data.

Next, we generate textual descriptions for each selected feature, appending a prefix to provide task context for the LLM. The transformed training data is then used to extract 20 text samples—ten labeled as 0 and ten labeled as 1. These labels are scaled by a factor of 10, mapping them to the range $[0, 10]$.

For inference, the transformed test samples are fed into the LLM along with additional textual guidelines. The LLM then returns a score from 1 to 10, where 1 indicates a patient was discharged alive and 10 signifies an in-hospital death. These scores are further normalized to a $[0, 1]$ range and evaluated using AUROC and AUPRC metrics.

We explored different strategies for incorporating training samples into each query for few-shot predictions. We tested both a fixed set of five samples and random sampling from a larger pool of 100 samples. However, providing a batch of 20 training samples in a single message before sequentially querying the LLM yielded better results. Consequently, this approach is used for the "with scaling and imputation" section of Table 4. For the "no scaling, no imputation" modality, using 20 separate training samples significantly degraded few-shot prediction performance. Therefore, we report results using the fixed set of five training samples.

	No scaling and imputation		With scaling and imputation	
	Zero-shot	Few-shot	Zero-shot	Few-shot
AuROC	0.5157	0.5090	0.5395	0.5568
AUPRC	0.1495	0.1489	0.1578	0.1667

Table 4: Performance comparison of Gemma-2b model with and without training examples.

By analyzing the experimental results with the LLM, we observe that it performs poorly in the mortality prediction task. The model achieves an AuROC score of approximately 0.5 and a low AUPRC of 0.15, indicating performance no better than random chance. However, applying the scaling and imputation methods developed in Q2.1.2 improves model performance compared to

simple feature aggregation without these techniques. Furthermore, incorporating training samples into the prompt (few-shot prediction) enhances performance in the case of scaled and imputed features.

Using LLMs to retrieve embeddings As shown in Table 5, LLM embeddings perform significantly worse than both supervised and representation learning. These results suggest that while LLM embeddings may capture some meaningful representations in a general case, they are not inherently optimized for this specific predictive task.

Upon visualizing the LLM embeddings using t-SNE (see Figure 3), we observe an equally bad clustering compared to the grouping of domain-specific pretrained embeddings. This reinforces the idea that LLM embeddings fail to capture domain-specific patterns as effectively, which contributes to their poorer performance in this task.

Using time-series foundation models We use the *ChronosPipeline* to compute embeddings for patient time-series data, where each patient’s data is aggregated across channels into a single vector representation. Since each channel’s time-series data may have a different sequence length, the embeddings for each channel are firstly reduced along the sequence dimension. After computing the mean embeddings for all channels of a patient, we aggregate these embeddings into a single patient-level embedding by taking the mean across all channels. These embeddings and the non-aggregated ones (before the aggregation across channels) are saved and later loaded to train a logistic regression model for predicting patient outcomes.

We use the pre-channel-aggregation embeddings as input for a neural network that learns to aggregate the data. We apply an attention layer to weight the importance of each channel and aggregate them into a single prediction. The model is trained using binary cross-entropy loss with early stopping to prevent overfitting. Comparing the performance of both aggregation methods, we observe that the attention-based aggregation outperforms simple aggregation but still performs worse than the simpler methods created in the beginning.

	LLM embeddings	Simple aggr.	Attention-based aggr.	SL	RL
AuROC	0.5	0.6345	0.7123	0.843	0.8263
AuPRC	0.1463	0.2362	0.3136	0.522	0.4704

Table 5: Performance comparison of linear probe on LLM embeddings, non-parametric pooling (Simple aggr.), neural network based aggregation strategies (Attention-based aggr.), supervised learning with Random Forest and feature engineering (Section 2), and representation learning with Logistic Regression on Embeddings,

5 General Questions

Q5.1 Yes, we have observed that the Random Forest model where we imputed the data with last measured value has outperformed the other models such as LSTMs and Transformers. The reason why Random Forest performed better might be because it handles data structured in a tabular form better than the other models. LSTMs, Transformers and LLMs usually work on sequences, and they are mainly used for forecasting.

Q5.2 The attention mechanism layer in a standard Transformer has a computational complexity of $O(n^2 * d)$ [2], where n is the sequence length and d is the representation dimension. As we can see, for very long sequences, the number of computations in the attention mechanism block grows quadratically, which can increase the cost of training the model and doing inference. A recurrent layer has a complexity of $O(n * d^2)$ [2], so RNN-based models scale linearly.

Based on the complexity, LSTM seem to be the choice for very long sequences. But LSTMs have to be trained sequentially, whereas Transformers can be trained in parallel using GPUs. Furthermore, Transformer can capture long-range dependencies significantly better than RNN based models. Hence, conceptually, a Transformer is more suitable for long time series data.

Q5.3 A major challenge in contrastive learning is the alignment problem, especially with ICU time series data. Unlike images, where augmentations like rotation or cropping work well, ICU data has strong temporal dependencies, making it hard to apply meaningful transformations. If the data is not independently and identically distributed, contrastive learning can also struggle.

We found that many common augmentations distort important clinical patterns, hurting the model’s performance. Although we expected representation learning to work better than supervised models with limited labels, poor augmentations may have prevented it from capturing useful features. Additionally, self-supervised methods are very sensitive to hyperparameter tuning, making it harder to improve the model. Lastly, while t-SNE and UMAP plots help visualize clustering, using them to make meaningful model adjustments is non-trivial.

6 Appendix

6.1 Data Transformation

First of all, we process the raw patient data, converting time into hourly intervals, removing duplicates if two values are recorded for the same hourly interval, and pivoting the data to ensure each patient has a consistent structure with 49 time points (0–48 hours). Static variables such as age and gender are cleaned (-1 values are replaced by Nan), propagated across all time points, and missing values are replaced by Nan. The processed data is saved as a Parquet file for each set.

To extract the labels, we simply read the outcome file and extract the in-hospital mortality labels into Parquet files for each set.

References

- [1] msundarv. “Physionet-Cardiology-Challenge-2012”. In: (2019). URL: <https://github.com/Msundarv/Physionet-Cardiology-Challenge-2012>.
- [2] Ashish Vaswani et al. “Attention Is All You Need”. In: (2023). arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.
- [3] Ailing Zeng et al. “Are Transformers Effective for Time Series Forecasting?” In: (2022). arXiv: 2205.13504 [cs.AI]. URL: <https://arxiv.org/abs/2205.13504>.